

Sistemas Operativos

Sincronização e Programação Concorrente

1

1

Revisão de conceitos

Multiprogramação

- Um processador que é partilhado por vários processos

Multiprocessamento

- Vários processadores que trabalham em conjunto (2+)
 - Inclui sistemas multicore
- Vantagens
 - Mais eficiente e processamento mais rápido (execução em **paralelo**)
 - Mais confiável (se um falha, há outro)
 - Mais complexo...
 - ...necessidade de interagir e **sincronizar** interações

2

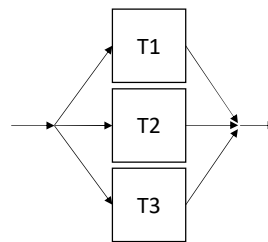
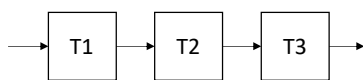
2

Revisão de conceitos

Os processadores multicore têm várias unidades de processamento dentro de um mesmo chip

Será que uma máquina multicore é sempre mais rápida que uma single core, considerando a mesma velocidade de processador?

- Depende... sequencial vs multi-thread



3

3

Programação Concorrente – Exemplos

- Operação sequencial:

for (j = 1; j <= 3; j++)

- $a(j) = b(j) + c(j);$

- Operação concorrente:

- Processador #1
 - $a(1) = b(1) + c(1)$
- Processador #2
 - $a(2) = b(2) + c(2)$
- Processador #3
 - $a(3) = b(3) + c(3)$

4

4

Programação Concorrente – Exemplos

$$A = 3 * B * C + 4 / (D + E)^{(F-G)}$$

Step	Operation	Result
1	$(F - G)$	$T1$
2	$(D + E)$	$T2$
3	$T2^{T1}$	$T1$
4	$4/T1$	$T2$
5	$3 * B$	$T1$
6	$T1 * C$	$T1$
7	$T1 + T2$	A

Step	Processor	Operation	Result
1	1	$3 * B$	$T1$
	2	$(D + E)$	$T2$
	3	$(F - G)$	$T3$
2	1	$T1 * C$	$T4$
	2	$T2^{T3}$	$T5$
3	1	$4/T5$	$T1$
4	1	$T4 + T1$	A

5

5

Programação Concorrente – Exemplos

Pessoa A

9:00 Acorda e não há pão
 9:05 Desloca-se à padaria
 9:10 Chega à padaria
 9:15 Sai da padaria
 9:20 Volta a casa e guarda o pão
 9:25
 9:30

Pessoa B

Acorda e não há pão
 Desloca-se à padaria
 Chega à padaria
 Sai da padaria
 Volta a casa e afinal há pão!!

6

6

Programação Concorrente – Exemplos

- Cenário: 2 levantamentos de dinheiro de uma mesma conta bancária

Tarefa A

```
if(saldo >= valor)
```

```
    saldo = saldo – valor
```

Tarefa B

```
if(saldo >= valor)
```

```
    saldo = saldo – valor
```

Levanta mais
do que tem?

7

7

Programação Concorrente – Exemplos

- Cenário: 2 levantamentos de dinheiro de uma mesma conta bancária

Tarefa A

```
if(saldo >= valor)
```

```
    saldo = saldo – valor
```

Tarefa B

```
if(saldo >= valor)
```

```
    saldo = saldo – valor
```

8

8

Programação Concorrente – Exemplos

- Exemplo de fila de processos a executar: a) ver qual o próximo na lista; b) Carrega esse processo; c) atualiza a lista; d) executa o processo que retirou da lista.

Processador A	Processador B	
1 a) Vê qual o 1º da lista		54
2 b) Carrega o 54		55
3		56
4	a) Vê qual o 1º da lista	57
5	b) Carrega o 54	...
6 c) Atualiza a lista (56 para o topo)	c) Atualiza lista (55 para o topo)	
7 d) Executa o 54		
8	d) Executa o 54	

...O que acontece
ao 54? E ao 55?

9

9

Programação Concorrente – Exemplos

```
void send_to_printer(int user_id, char* document)
{
    printer_write("Job from user --- %d ---", user_id);
    printer_write("%s", document);
    printer_write("-----");
}
```

Process A

userID 50

send_to_printer(50, "What a beautiful day!");

Process B

userID 12

send_to_printer(12, "I hate going to school!");

10

10

Programação Concorrente – Exemplos

- Outputs possíveis:

```
Job from user --- 50 ---  
What a beautiful day!  
-----
```

```
Job from user --- 12 ---  
I hate going to school!  
-----
```

Resultado desejado

```
Job from user --- 50 ---  
Job from user --- 12 ---  
What I hate a beautiful day!  
-----
```

```
going to school!  
-----
```

Resultado possível

!Condição de corrida (race condition)!: vários processos competem de forma concorrente pelo acesso a um recurso partilhado (resultado depende dos momentos em que cada um chega)

11

11

Situações Típicas de Sincronização

1. Sincronização de 2+ tarefas que competem pelo **acesso a um recurso partilhado**
 - Exclusão Mútua (execução de forma indivisível – atómica)
 - Secção Crítica (bloco de código ao qual apenas 1 tarefa de cada vez pode aceder)
 - Lock() [bloco da secção crítica] Unlock()
2. Sincronização entre tarefas que cooperam na **execução de uma atividade comum** (variáveis de condição e semáforos ...vamos aprofundar mais tarde)

12

12

Requisitos de uma Secção Crítica [Disjkstra 1965]

1. Assegurar **Exclusão Mútua**: só pode estar dentro da secção crítica 1 tarefa de cada vez
2. Assegurar **Progresso**: se a secção crítica está 'livre' e há tarefas que querem aceder, então uma delas tem que o conseguir (eliminar deadblock)
3. Eliminar **Starvation**: uma tarefa que pretende entrar na secção crítica não ficará sistematicamente impedida de o fazer (não pode descriminar e priorizar)
4. Assegurar **Eficiência**: se apenas uma tarefa pretende entrar na secção crítica, deve conseguir de forma rápida e eficiente

13

13

Exclusão Mútua Algorítmica

- Situação: realizar levantamentos numa mesma conta

Tarefa A

...

Tarefa B

...

- Vamos ter acesso a um recurso partilhado...

- ...como podemos controlar os acessos?

14

14

Exclusão Mútua Algorítmica (1ª tentativa)

- Acesso a um recurso partilhado: saldo e trinco

Tarefa A

```
while (trinco == fechado)
{ //loop vazio };
```

```
trinco = fechado
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trinco = aberto
```

Tarefa B

```
while (trinco == fechado)
{ //loop vazio };
```

```
trinco = fechado
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trinco = aberto
```

Funciona?
Respeita requisitos?

15

15

Exclusão Mútua Algorítmica (2ª tentativa)

- Acesso a um recurso partilhado: saldo e trincoA e trincoB

Tarefa A

```
trincoA = fechado
```

```
while (trincoB == fechado)
{ //loop vazio };
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trincoA = aberto
```

Tarefa B

```
trincoB = fechado
```

```
while (trincoA == fechado)
{ //loop vazio };
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trincoB = aberto
```

Funciona?
Respeita requisitos?

16

16

Exclusão Mútua Algorítmica (3ª tentativa)

- Acesso a um recurso partilhado: saldo e trincoVez

Tarefa A

```
while (trincoVez == B)
{ //loop vazio };
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trincoVez = B
```

Tarefa B

```
while (trincoVez == A)
{ //loop vazio };
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trincoVez = A
```

Funciona?
Respeita requisitos?

17

17

Exclusão Mútua Algorítmica (PETERSON)

- Desempatar a interblocagem dando **prioridade** a 1 das 2 tarefas.
trincoA; trincoB; **tarefaPrio**

Tarefa A

```
trincoA = fechado  
tarefaPrio = B
```

```
while (trincoB == fechado  
      && tarefaPrio == B)
{ //loop vazio };
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trincoA= aberto
```

Tarefa B

```
trincoB = fechado  
tarefaPrio = A
```

```
while (trincoA == fechado  
      && tarefaPrio == A)
{ //loop vazio };
```

```
if(saldo >= valor)
    saldo = saldo - valor
```

```
trincoB= aberto
```

Funciona?
Respeita requisitos?
Qual o problema?

18

18

Exclusão Mútua (outras soluções)

- PETERSON

- Desempatar a interblocagem (deadlock) dando prioridade a 1 das 2 tarefas
 - trincoA; trincoB; tarefaPrio
 - Só funciona com 2 tarefas...

- LAMPORT (não vamos detalhar)

- Dá resposta a N tarefas atribuindo uma senha a cada uma

- Baseada em Hardware (não vamos detalhar)

- Inibição de interrupções (o lock inibe as interrupções e o unlock restaura)
 - Não funciona em multiprocessadores (cada inibição é aplicada àquele processador)

19

19

Exclusão Mútua (outras soluções)

!!ATENÇÃO!!:

- Estas soluções baseiam-se em **‘Espera Ativa’!** (proibido!)
 - `while (trinco == fechado){ //loop vazio };`
 - Vai manter-se a testar **repetidamente** o estado do `trinco` apesar da libertação deste `trinco` depender de outra tarefa (que naquele momento não pode usar o processador)... abordagem **muito ineficiente. A evitar...**
- Vamos ver o objeto do sistema operativo denominado **MUTEX** (**MUT**ual **EX**clusion) e como usá-lo para responder a estes problemas

20

20

Exclusão Mútua – MUTEX

Mutex

- Permitem exclusão mútua e definir secções críticas entre tarefas (processos também podem usar caso se esteja a usar memória partilhada)
- Evita-se a 'Espera Ativa': ao tentar fazer lock() dum mutex que já está fechado, a tarefa fica suspensa na fila de espera do mutex até ser 'notificada' que o mutex já abriu (em vez de ficar ativa em loop a verificar repetidamente o estado do trinco)
- Quem faz lock() do mutex é quem deve fazer unlock()
- Se uma tarefa tenta fazer unlock() de um mutex que foi outra tarefa a fazer o lock() – ou que estava já unlocked – o resultado é indefinido

21

21

Exclusão Mútua – MUTEX

- Acesso a um recurso partilhado: saldo

Tarefa A

Lock(idmutex)

```
if(saldo >= valor)  
    saldo = saldo - valor
```

Unlock(idmutex)

secção crítica

Tarefa B

Lock(idmutex)

```
if(saldo >= valor)  
    saldo = saldo - valor
```

Unlock(idmutex)

22

22

Instruções Mutex

```
pthread_mutex_t name_lock; //declare lock
int pthread_mutex_init(pthread_mutex_t *mutex, pthread_mutexattr_t *attr); //initialize
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- Exemplo de uso:

```
pthread_mutex_t abcLock;
pthread_mutex_init(&abcLock, NULL);

pthread_mutex_lock(&abcLock);
var++;
pthread_mutex_unlock(&abcLock);

pthread_mutex_destroy(&abcLock);
```

23

23

MUTEX – Exemplo demomutex.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define RUNS 1000000

pthread_t tid[2];
long counter = 0;
pthread_mutex_t countLock;

void* increment()
{
    for(long i=0; i<RUNS; i++)
    {
        //I am about to change a shared variable...
        //??? add code...
        counter = counter + 1;
        //???

        usleep(10); //simulate some work
    }

    pthread_exit(NULL);
    return NULL;
}

int main(void)
{
    /* mutex is declared, initialized and destroyed.
     * Run the program to check its behaviour and
     * complete the code to fix it
     */
    if (pthread_mutex_init(&countLock, NULL) != 0)
    {
        printf(":::MUTEX error on init\n");
        return 3;
    }

    printf("Counter initial value: %ld\n", counter);

    if (pthread_create(&tid[0], NULL, &increment, NULL) != 0)
    {
        printf(":::Thread [0] error on create\n");
    }

    if (pthread_create(&tid[1], NULL, &increment, NULL) != 0)
    {
        printf(":::Thread [1] error on create\n");
    }

    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);

    printf("Counter final value: %ld\n", counter);

    pthread_mutex_destroy(&countLock);

    return 0;
}
```

24

24

MUTEX – Exemplo demomutex.c

No código apresentado temos uma variável global `counter` que inicia com valor 0. De seguida são lançadas 2 tarefas: cada uma vai incrementar `counter` 1M de vezes. Espera-se, portanto, que o valor final de `counter` seja 2M.

1. Analise o código
2. Compile e execute (para verificar o comportamento errado pode ser necessário aumentar o número de RUNS)
3. Analise o output
4. Acrescente o código necessário para definir secções críticas e corrigir o comportamento do programa

25

25

Sistemas Operativos

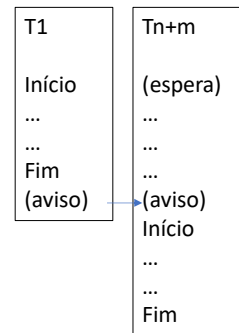
Semáforos

26

26

Semáforos

- Motivação: podemos querer permitir as primeiras n tarefas a entrar dentro de uma secção, bloqueando as que chegarem depois (e não apenas 1 de cada vez)
- Se definir $n=3$, precisamos de um mecanismo que:
 - Permita executar as 3 primeiras tarefas e que bloqueie as seguintes
 - Permita, ao terminar 1 das tarefa, avisar uma das bloqueadas que já pode prosseguir
- Já tínhamos visto casos de espera ativa? (a evitar) e MUTEXs



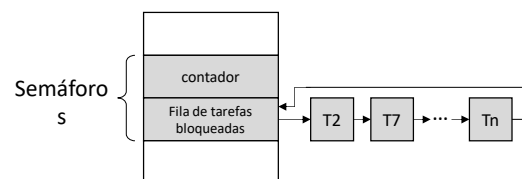
27

27

Semáforos

Solução: Semáforo

- É um objeto do SO com 2 atributos:
 - Variável de controlo (contador*)
 - Fila de tarefas bloqueados no semáforo
- E com as chamadas de sistema:
 - **Esperar (wait)**
 - se o contador $cont \leq 0$ bloqueia a tarefa que chamou
 - senão decrementa o contador e continua ($cont = cont - 1$)
 - **Assinalar (post)**
 - se houver tarefas bloqueadas então retira 1 da fila e passa-a para execução
 - senão incrementa o contador e continua ($cont = cont + 1$)

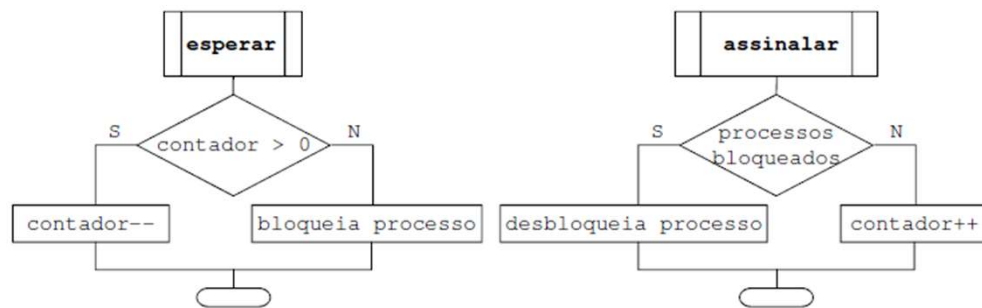


28

28

Semáforos

Solução: Semáforo



29

29

Semáforos

- Existem 2 tipos:
 - **unnamed** (entre tarefas – ou entre processos se for usada memória partilhada)
 - **named** (entre processos, onde cada processo se pode associar ao semáforo através do nome)
- Podem ser usados como:
 - semáforos binários
 - semáforos de contagem (de recursos finitos)

30

30

Instruções Semáforos (unnamed)

```
sem_t sem_var; //declare semaphore  
int sem_init(sem_t *sem, int pshared, unsigned int value);  
int sem_wait(sem_t *sem);  
int sem_post(sem_t *sem);  
int sem_destroy(sem_t *sem);
```

Exemplos de uso:

```
sem_t semB;  
sem_init(&semB, 0, 10); //counter initial value =10  
sem_wait(&semB);  
sem_post(&semB);  
sem_destroy(&semB);
```

31

31

Instruções Semáforos (named)

```
sem_t *sem;  
sem_t *sem_open(char *name, int oflag, mode_t mode, int value); //creates or uses an existing sem  
int sem_wait(sem_t *sem);  
int sem_post(sem_t *sem);  
int *sem_close(sem_t *sem);  
int *sem_unlink(char *name); //removes and destroys after last open reference is closed
```

Exemplos de uso:

```
sem_t *semptr; //sem pointer  
sem = sem_open("SEM_NAME", O_CREAT | O_EXCL, SEM_FLAGS, 1);  
sem_wait(semptr);  
sem_post(semptr);  
sem_close(semptr);  
sem_unlink("SEM_NAME");
```

32

32

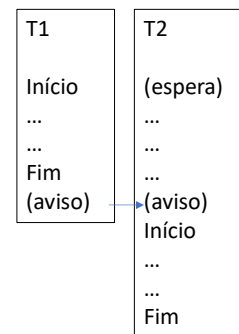
Semáforos

- Assuma-se que só pode estar 1 tarefa de cada vez dentro de determinada secção

- `sem_init(&sem_name, 0, 1);*`

```
T1
sem_wait(&sem_name);
Início
...
...
Fim
sem_post(&sem_name);
```

```
T2
sem_wait(&sem_name);
(Bloqueada)
...
...
Início
...
...
Fim
sem_post(&sem_name);
```



33

33

Semáforos

- Assuma-se que só podem estar 2 tarefas de cada vez dentro de determinada secção

- `sem_init(&sem_name, 0, 2);*`

```
T1
sem_wait(&sem_name);
Início
...
...
Fim
sem_post(&sem_name);
```

```
T2
sem_wait(&sem_name);
Início
...
...
Fim
sem_post(&sem_name);
```

```
T3
sem_wait(&sem_name);
(Bloqueada)
...
...
Início
...
...
Fim
sem_post(&sem_name);
```

34

34

Exclusão Mútua – pthread_mutex_t Vs. sem_t

- Podemos implementar exclusão mútua com semáforos?

```
pthread_mutex_t trinco;  
pthread_mutex_init(& trinco, NULL);  
  
pthread_mutex_lock(&trinco)  
if(saldo >= valor)  
    saldo = saldo - valor  
pthread_mutex_unlock(&trinco)  
  
pthread_mutex_destroy(&trinco);
```

secção crítica

35

35

Exclusão Mútua – pthread_mutex_t Vs. sem_t

- Podemos implementar exclusão mútua com semáforos?

```
pthread_mutex_t trinco;  
pthread_mutex_init(& trinco, NULL);  
  
pthread_mutex_lock(&trinco)  
if(saldo >= valor)  
    saldo = saldo - valor  
pthread_mutex_unlock(&trinco)  
  
pthread_mutex_destroy(&trinco);
```

secção crítica

```
sem_t sem_trinco;  
sem_init(&sem_trinco, 0, 1);  
  
sem_wait(&sem_trinco)  
if(saldo >= valor)  
    saldo = saldo - valor  
sem_post(&sem_trinco)  
  
sem_destroy(&sem_trinco);
```

Podemos revisitar o exercício do slide 24,
mas utilizando agora 1 semáforo BINÁRIO

36

36

Semáforos

- Com base no exercício anterior, crie um programa que lança 4 tarefas. As tarefas devem:
 - 1) imprimir uma frase a dizer “entrei na secção crítica”;
 - 2) invocar um sleep de 3 segundos;
 - 3) imprimir uma frase a dizer “saí da secção crítica”.
- Com recurso a um semáforo, limite a execução a apenas 3 tarefas em simultâneo (das 4 lançadas).
- O output deverá ser semelhante ao seguinte:

```
Starting main
I'm inside critical section
I'm inside critical section
I'm inside critical section
I'm exiting critical section
I'm exiting critical section
I'm inside critical section
I'm exiting critical section
I'm exiting critical section
Exiting main
```

37

37

Conceitos a manter

- Deadlock (ou interblocagem)
 - Quando 2 ou + processos não conseguem prosseguir porque estão a ser bloqueados um pelo outro. Os processos detêm recursos que os impede mutuamente de prosseguir(estão todos no estado 'bloqueado')
- Starvation (ou míngua)
 - Quando um processo não consegue aceder a um recurso que necessita para progredir (é indefinidamente preterido no acesso a uma região crítica)
- Nunca interbloquear semáforos!
 - Os semáforos devem ser adquiridos pela mesma ordem em todos os processos (sempre que possível)
 - Os semáforos devem ser libertados em ordem inversa à que foram adquiridos

38

38

Sistemas Operativos

Variáveis de Condição

39

39

Variáveis de Condição

Mutex

- Permitem exclusão mútua e definir secções críticas entre tarefas (processos também podem usar caso se esteja a usar memória partilhada)
- Semelhante ao semáforo binário

Semáforos (binários e de contagem)

- unnamed (entre tarefas – ou entre processos se for usada memória partilhada)
- named (entre processos, onde cada processo se pode associar ao semáforo através do nome)
- Permitem assinalar eventos ou contar objetos de forma sincronizada

Variáveis de Condição

- Alternativa aos semáforos
- Permitem que uma tarefa espere por uma condição que depende da ação de outra tarefa
- Tem um mutex associado
- Pode haver mais que uma variável de condição associada ao mesmo trinco

40

40

Instruções Variáveis de Condição

```
pthread_cond_t name;
pthread_cond_init(pthread_cond_t *name, const pthread_condattr_t *attr);
pthread_cond_wait(pthread_cond_t *name, pthread_mutex_t *mutex); //mutex é libertado enquanto espera e readquirido quando for desbloqueada
pthread_cond_signal(pthread_cond_t *name); //se houver tarefas na fila, debloqueia 1 (se não houver não tem efeito!)
pthread_cond_broadcast(pthread_cond_t *name); //notifica todas as tarefas na fila
pthread_cond_destroy(pthread_cond_t *name);
```

- Exemplo de uso:

```
pthread_mutex_t mutex;
pthread_cond_t cv;
pthread_cond_init(&cv, NULL);
pthread_cond_wait(&cv, &mutex);
pthread_cond_signal(&cv);
pthread_cond_destroy(&cv);
```

41

41

Variáveis de Condição

Regras:

- Tem sempre um **mutex associado**
- A condição deve ser **sempre verificada em exclusão mútua (mutex lock)**
- Quando a tarefa faz **wait**, o mutex é **automaticamente libertado (mutex unlock)**
- Quando o wait for **desbloqueado**, o mutex é **automaticamente readquirido (mutex lock)**

T1

```
pthread_mutex_lock(&count_mutex);
while (count < COUNT_LIMIT) {
    //wait liberta o trinco e fica em espera
    pthread_cond_wait(&count_limit_cv, &count_mutex);
}
pthread_mutex_unlock(&count_mutex);
pthread_exit(NULL);
```

Desbloqueia o wait para voltar a verificar se a condição já é verdadeira.
Porquê ciclo while?

T2

```
pthread_mutex_lock(&count_mutex);
count++;
if (count == COUNT_LIMIT) {
    //signal acorda o wait para voltar a verificar a condição
    pthread_cond_signal(&count_limit_cv);
}
pthread_mutex_unlock(&count_mutex);
```

42

42

Variáveis de Condição – Exemplo prático

Código: `aula04slide86demovariaveiscondicionais.c`

- São lançadas 3 threads
- Duas delas vão incrementar, de forma concorrente, um contador (count)
- A outra vai ficar à espera até que o contador atinja um valor limite estipulado

Analise e execute o código fornecido. Verifique se o output produzido é o esperado tendo em conta o código.